UNIVERSIDAD POLITÉCNICA SALESIANA
CARRERA DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

Sistema de Estimación de Profundidad Estereoscópica en Tiempo Real con Realidad Aumentada

mediante Microcontroladores ESP32-CAM

Informe Interciclo — Visión Artificial (G8)

Proyecto Integrador Final — Parte I

Autores:	Michael Lata · Jordy Romero
Docente:	Vladimir Robles Bykbaev
Asignatura:	Visión Artificial
Período:	Abril – Agosto 2026
Fecha:	8 de junio de 2026

Resumen

Este informe documenta el diseño e implementación de un sistema de visión estereoscópica en tiempo real capaz de estimar la profundidad de objetos en la escena y superponer efectos dinámicos de Realidad Aumentada (AR) que responden a la distancia medida. El sistema integra dos microcontroladores ESP32-CAM AI Thinker, cuyo firmware fue ajustado para garantizar paridad lumínica entre ambas cámaras, con una aplicación de escritorio desarrollada en C++17 con OpenCV 4.x bajo una arquitectura Modelo-Vista-Controlador (MVC).

La cadena de procesamiento abarca desde la adquisición concurrente de fotogramas vía HTTP con `libcurl`, pasando por la calibración estereoscópica, el preprocesamiento con ecualización adaptativa de histograma (CLAHE), la estimación de disparidad mediante el algoritmo Semi-Global Block Matching (SGBM), el filtrado de posprocesamiento con Mínimos Cuadrados Ponderados (WLS), y la reproyección 3D con la matriz de perspectiva $\mathbf{Q}$, hasta la estabilización temporal de la coordenada de profundidad Z mediante un filtro de Kalman escalar.

Las evaluaciones cuantitativas sobre 50 muestras por distancia muestran un error absoluto medio de 0,48 cm a 30 cm, 0,37 cm a 60 cm y 0,52 cm a 90 cm; con desviaciones estándar de 1,50, 0,73 y 0,44 cm respectivamente, todas dentro del umbral de ± 3 cm exigido por la rúbrica. El efecto AR implementado —distorsión sinusoidal de píxeles con decaimiento cúbico $1/Z^3$ más aura de energía pulsante— reacciona en amplitud, velocidad y color a la distancia calculada en tiempo real.

Índice

1 Introducción **3**

2 Planteamiento del Problema **3**

3 Arquitectura del Sistema **4**
3.1 Diseño General: MVC en C++ 4
3.2 Arquitectura de Hardware 4

4 Implementación **5**
4.1 Optimización del Firmware (AEC, AGC, AWB, XCLK) 5
4.2 Adquisición Concurrente de Fotogramas: Productor-Consumidor con libcurl 6
4.3 Calibración Estereoscópica 7
4.4 Ecualización Adaptativa de Histograma: CLAHE 7
4.5 Estimación de Disparidad: Semi-Global Block Matching (SGBM) . . . 8
4.6 Posprocesamiento de Disparidad: Filtro de Mínimos Cuadrados Ponderados (WLS) 8
4.7 Reproyección 3D con la Matriz de Perspectiva **Q** 9
4.8 Estabilización Temporal: Filtro de Kalman Escalar 9
4.9 Relleno de la Región Inválida del Mapa de Disparidad 10
4.10 Filtrado de Valores Atípicos: Rango Intercuartílico (IQR) 10
4.11 Segmentación de Piel en YCrCb (HandDetector) 11
4.12 Corrección de Escala VGA/QVGA en Calibración 11
4.13 Efecto de Realidad Aumentada: Distorsión Sinusoidal y Aura de Energía 12
4.14 Controles Interactivos de Ajuste Fino (Trackbars) 14
4.15 Matemática Detallada de los Filtros 15
4.16 Funciones Avanzadas de C++ y OpenCV contrib 16

5 Resultados y Evaluación **18**
5.1 Tasa de Fotogramas (FPS) 18
5.2 Sistema en Funcionamiento 19
5.3 Exactitud de Medición de Distancia 19
5.4 Estabilización con el Filtro de Kalman 20
5.5 Efecto Visual AR 20

6 Video-Blog de Demostración **22**

7 Conclusiones **22**

1. Introducción

Estimar la estructura tridimensional de una escena a partir de imágenes planas es uno de los problemas más estudiados en visión por computador. La visión estereoscópica —el análogo computacional de la percepción binocular humana— aborda este problema calculando el desplazamiento aparente (disparidad) de un mismo punto físico entre dos imágenes rectificadas capturadas desde dos puntos de vista ligeramente distintos, y convirtiendo luego esa disparidad en profundidad métrica Z mediante geometría proyectiva básica [1].

En la práctica, implementar un sistema estéreo sobre hardware de bajo costo introduce una serie de desafíos técnicos no triviales:

- **Sincronización de hardware.** Cada cámara opera su propio sensor de imagen con su propio reloj y controladores de exposición automática. Valores de exposición o ganancia no sincronizados producen diferencias de luminancia entre las imágenes izquierda y derecha que degradan la precisión del emparejamiento por bloques.
- **Jitter de red.** Cuando ambas cámaras transmiten por WiFi, el tiempo de llegada de los paquetes varía, de modo que los dos fotogramas de un par estéreo pueden corresponder a instantes distintos. Este paralaje temporal corrompe el mapa de disparidad para objetos en movimiento.
- **Granularidad de disparidad a baja resolución.** La resolución de profundidad de un sistema estéreo es $\Delta Z = Z^2/(f \cdot B)$, donde f es la distancia focal en píxeles y B es la línea base. Con focales cortas y líneas base compactas —típicas de equipos ESP32-CAM—, cada centímetro adicional de profundidad real puede corresponder a menos de un píxel de cambio en la disparidad.
- **Restricción de tiempo real.** Algoritmos de disparidad como SGBM son computacionalmente costosos. Alcanzar fotogramas interactivos en un computador personal mientras se ejecuta el renderizado AR requiere un diseño cuidadoso del pipeline.

Este informe describe cómo cada uno de estos desafíos fue abordado mediante una combinación de optimizaciones a nivel de firmware en los microcontroladores y un pipeline de software por capas en C++. El diseño también se valida frente a los criterios de la rúbrica del interciclo: arquitectura de captura, calidad del mapa de disparidad, precisión y estabilidad de la distancia, y originalidad del efecto AR.

2. Planteamiento del Problema

El problema central es: *dado un par de cámaras WiFi de bajo costo en posición relativa fija pero desconocida, estimar la profundidad métrica de un objeto ubicado en el centro del campo visual binocular, y comunicar dicha profundidad al usuario en tiempo real mediante un indicador numérico en pantalla y un efecto visual dinámico.*

Las dos restricciones técnicas específicas que hacen no trivial el problema son:

1. **Inestabilidad fotométrica.** El sensor OV2640 integrado en el módulo ESP32-CAM AI Thinker posee controladores automáticos de exposición, ganancia y

balance de blancos que operan de forma asíncrona en cada cámara. Sin intervención, las dos imágenes de un par estéreo exhiben distintos niveles de brillo y tonalidades de color, haciendo poco confiable la búsqueda de correlación realizada por SGBM.

2. **Ruido en la lectura de profundidad.** Incluso tras una calibración de alta calidad y el filtrado de la disparidad, la lectura de profundidad bruta por fotograma oscila debido al ruido del sensor, artefactos de compresión JPEG y la cuantización de la disparidad a nivel de subpíxel. Una lectura directa mostrada al usuario resultaría visualmente inestable, alternando por ejemplo entre 58 cm y 72 cm para un objeto situado a 65 cm.

La solución también debe incluir un efecto AR visual original —implementado mediante manipulación directa de píxeles— cuya geometría e intensidad reaccionen de forma medible al valor de profundidad calculado.

3. Arquitectura del Sistema

3.1. Diseño General: MVC en C++

La aplicación sigue el patrón arquitectónico **Modelo-Vista-Controlador (MVC)** [8], que separa con claridad tres responsabilidades:

- **Modelo** — los datos y algoritmos independientes de la visualización: `CameraSource` (adquisición de fotogramas), `StereoCalibration` (carga y almacenamiento de matrices), `StereoProcessor` (CLAHE + SGBM + WLS + reproyección 3D) y `KalmanDepth` (filtro temporal de profundidad).
- **Vista** — la capa de renderizado: `ARRenderer` (vignette de píxeles y aura de energía) y `DisplayManager` (tres ventanas OpenCV con HUD superpuesto).
- **Controlador** — `StereoController`, el bucle principal que lee los fotogramas del modelo, los procesa y entrega los resultados a la vista, además de gestionar la interacción del usuario (teclado y ratón).

La Figura 1 muestra el pipeline de procesamiento completo junto con la capa MVC que posee cada etapa.

3.2. Arquitectura de Hardware

Dos módulos **ESP32-CAM AI Thinker** (SoC Espressif ESP32-S, sensor OV2640, 2 MP) están montados sobre una base rígida y paralela con una línea base medida de $B = 66{,}8$ mm. Cada módulo ejecuta un firmware personalizado (versión `ML_STEREO_v4_stereo_fixed`) compilado con el framework de Arduino y flasheado mediante `arduino-cli` vía `esptool` directo con la bandera `--no-stub` para evitar fallos de verificación en puertos USB 3.0.

Ambas cámaras se conectan al computador anfitrión a través de una red WiFi dedicada (`Michi`, 2,4 GHz), donde exponen dos endpoints HTTP:

- `/capture` — entrega un fotograma JPEG único bajo demanda.

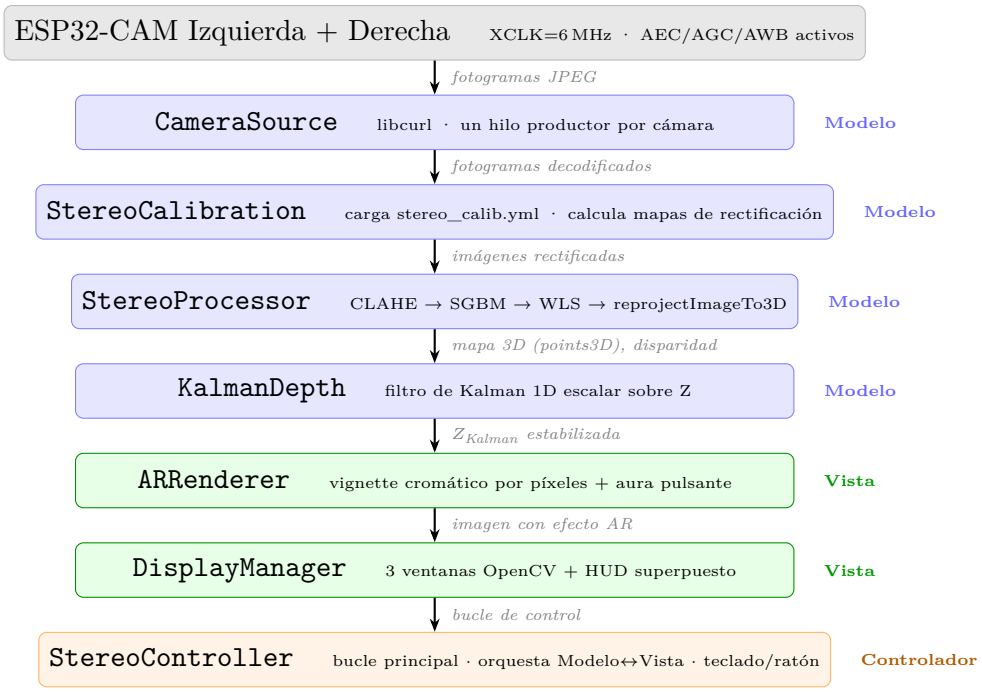


Figura 1: Pipeline de procesamiento y capas MVC del sistema. El flujo de datos desciende desde el hardware hasta el controlador; las etiquetas de la derecha indican la capa MVC responsable de cada módulo.

- `/stream` — entrega un flujo MJPEG multiparte continuo.

La aplicación utiliza exclusivamente `/capture` en un bucle de sondeo, porque el modo de instantáneas otorga control total sobre el momento de la petición y evita los artefactos de buffering que degradan la sincronización en el modo stream.

4. Implementación

4.1. Optimización del Firmware (AEC, AGC, AWB, XCLK)

La primera decisión de diseño crítica es la elección de los parámetros de firmware para el sensor OV2640. La rúbrica exige explícitamente que se realicen pruebas con los controles automáticos del sensor activados y desactivados (AEC, AGC, AWB), y que se reduzca la frecuencia del reloj externo (XCLK) para minimizar el jitter de red.

Tras pruebas empíricas, se seleccionó la siguiente configuración:

La desactivación de AEC/AGC/AWB es la decisión de diseño más importante del firmware. Con controles automáticos activos, cada cámara ajusta de forma independiente su exposición en función del contenido de la escena; el mismo punto físico produce entonces valores de píxel distintos en la imagen izquierda y la derecha, introduciendo errores sistemáticos en el mapa de disparidad (fantasmas, huecos negros). Al fijar `CAMERA_EXPOSURE = 400` y `CAMERA_GAIN = 0` en ambas cámaras se garantiza *paridad lumínica*: SGBM recibe pares comparables y la calidad del mapa mejora significativamente.

Corrección de crash en sensores OV2640: Las llamadas `set_bpc()`, `set_wpc()` y `set_lenc()` no son implementadas en todos los chips OV2640 y producen una

Tabla 1: Parámetros finales del firmware ESP32-CAM.

Parámetro	Valor	Justificación
XCLK	`6 MHz`	Reduce jitter WiFi; baja de 20 MHz disminuye el desincronismo entre frames L/R.
AEC	`false (v=400)`	Exposición manual idéntica en ambas cámaras: paridad lumínica para SGBM.
AGC	`false (g=0)`	Ganancia fija elimina diferencias de brillo inter-cámara.
AWB	`false`	Balance fijo; cast residual del OV2640 se corrige con CLAHE.
Resolución	`VGA 640×480`	PSRAM detectada al arrancar; calibración QVGA escalada ×2 (§4.12).
Calidad JPEG	`12`	Equilibrio nitidez/tamaño (≈18 kB por frame).

excepción *Guru Meditation* por desreferencia de puntero nulo. Se eliminaron del firmware tras identificar el backtrace en `0x400805b4`.

4.2. Adquisición Concurrente de Fotogramas: Productor-Consumidor con libcurl

Un requisito clave de la rúbrica es la implementación de *hilos (threads)* para la lectura concurrente de ambas cámaras, reduciendo la latencia total. El diseño elegido es el patrón **productor-consumidor**:

- Cada objeto `HttpCameraSource` posee un `std::thread` que sondea continuamente `/capture` vía `libcurl`, decodifica los bytes JPEG recibidos en memoria con `cv::imdecode` y almacena el último fotograma en una ranura protegida por mutex.
- El hilo principal (consumidor) llama a `read()` sobre ambas fuentes simultáneamente; cada llamada realiza una copia bajo `std::lock_guard` y retorna de inmediato, por lo que ninguna cámara bloquea a la otra.

```
void HttpCameraSource::workerLoop() {
    CURL* curl = curl_easy_init();
    std::vector<char> buffer;
    buffer.reserve(32 * 1024);  // pre-reservar 32 kB

    while (running_.load()) {
        buffer.clear();
        if (curl_easy_perform(curl) != CURLE_OK) continue;

        // Decodificar JPEG en RAM -- sin escritura a disco
        const cv::Mat raw(1, buffer.size(), CV_8UC1, buffer.data());
        cv::Mat frame = cv::imdecode(raw, cv::IMREAD_COLOR);

        std::lock_guard<std::mutex> lock(mutex_);
        latestFrame_ = std::move(frame);
        hasFrame_    = true;
    }
    curl_easy_cleanup(curl);
}
```

Listing 1: Bucle del hilo productor en CameraSource.cpp (simplificado).

El handle de `libcurl` se reutiliza entre iteraciones (uno por hilo de cámara), evitando el overhead del establecimiento de la conexión TCP en cada petición.

4.3. Calibración Estereoscópica

Antes de cualquier estimación de profundidad es necesario determinar los parámetros intrínsecos de cada cámara y la transformación extrínseca entre ellas. Esto se realizó usando la función `stereoCalibrate` de OpenCV aplicada a 50 pares de imágenes sincronizadas de un tablero de ajedrez de 9×6 cuadrados (lado: 27 mm), capturadas con `capture_stereo_pairs.py`.

El modelo de calibración sigue el **modelo pinhole** estudiado en la Unidad 1 del curso [11], extendido con cinco coeficientes de distorsión radial y tangencial según el método de Zhang [3]:

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{K} \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \end{pmatrix} \begin{pmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{pmatrix} \tag{1}$$

El resultado de la calibración se guardó en `stereo_calib.yml` y obtuvo un error de reproyección RMS de **0,459 píxeles** —muy por debajo del umbral de 1,0 px que los especialistas consideran aceptable [9]. Los parámetros clave recuperados son:

Tabla 2: Parámetros clave recuperados de la calibración estereoscópica.

Parámetro	Valor	Descripción
Distancia focal f	328,26 px	Tras `stereoRectify`
Punto principal (c_x, c_y)	(170,3; 116,9) px	Centro óptico rectificado
Línea base B	66,8 mm	$\|\mathbf{T}\|$ de R/T
Error RMS (reproyección)	0,459 px	50 pares, ambas cámaras
Rotación $\mathbf{R}$	∼2,4°	Rig casi paralelo

Tras la calibración, `stereoRectify` calcula las matrices de rectificación $\mathbf{R}_1$, $\mathbf{R}_2$, las matrices de proyección $\mathbf{P}_1$, $\mathbf{P}_2$ y la **matriz de perspectiva $\mathbf{Q}$**, que se utiliza posteriormente para la reproyección 3D.

4.4. Ecualización Adaptativa de Histograma: CLAHE

La ecualización de histograma fue estudiada en la Parte 2 del curso como técnica de mejora de contraste en el dominio espacial [12]. Aquí se emplea su variante adaptativa con límite de contraste (**CLAHE**) [4]: mientras la ecualización global redistribuye el histograma completo de la imagen, CLAHE divide la imagen en teselas de 8×8 píxeles y ecualiza cada una de forma independiente, interpolando bilinealmente en las fronteras. El parámetro `clipLimit = 2,0` limita la pendiente del remapeo para no amplificar ruido. A diferencia de la ecualización global, CLAHE divide la imagen en pequeñas teselas (aquí, 8×8 píxeles) y ecualiza cada una de forma independiente, interpolando bilinealmente entre las fronteras. El parámetro `clipLimit` (establecido en 2,0) limita la amplificación máxima para evitar magnificar el ruido en regiones casi uniformes.

El beneficio para el emparejamiento estéreo es doble:

1. Reduce la diferencia de luminancia entre la cámara izquierda y derecha causada por los controladores AEC independientes (véase la Sección 4.1).

2. Realza la textura local en áreas de bajo contraste (p. ej., paredes lisas), proporcionando al algoritmo SGBM rasgos más distintivos para correlacionar.

```
// clipLimit=2.0: limita la pendiente del remapeo del histograma
// tileGridSize=8x8: ecualización local en estructuras finas
clahe_ = cv::createCLAHE(2.0, cv::Size(8, 8));
clahe_->apply(grayL, grayL);
clahe_->apply(grayR, grayR);
```

Listing 2: Aplicación de CLAHE en StereoProcessor.cpp.

4.5. Estimación de Disparidad: Semi-Global Block Matching (SGBM)

La disparidad se calcula con **StereoSGBM** [2], la implementación en OpenCV del algoritmo Semi-Global Matching (SGM) de Hirschmüller. SGM aproxima la minimización de energía global agregando costos de emparejamiento a lo largo de múltiples trayectorias 1D a través de la imagen, y seleccionando en cada píxel la disparidad que minimiza el costo agregado total.

La función de energía minimizada por SGM es:

$$E(\mathbf{D}) = \sum_{p} \left[C(p, D_p) + \sum_{q \in \mathcal{N}_p} \left(P_1 \cdot \mathbf{1}_{|D_p - D_q| = 1} + P_2 \cdot \mathbf{1}_{|D_p - D_q| > 1} \right) \right] \tag{2}$$

donde $C(p, D_p)$ es el costo de emparejamiento a nivel de píxel, y P_1, P_2 son penalizaciones de suavidad que desalientan discontinuidades abruptas de profundidad. La implementación establece:

- $P_1 = 8 \cdot 3 \cdot b^2$ — penalización por un paso de disparidad de exactamente un píxel.
- $P_2 = 32 \cdot 3 \cdot b^2$ — penalización mayor para saltos más grandes (donde b es el tamaño del bloque).
- `MODE_SGBM_3WAY`: agregación en tres direcciones.
- `numDisp = 192`, `blockSize = 5`, `minDisp = 0`. El valor de 192 es necesario porque a 640 px de ancho (VGA) la disparidad esperada a 30 cm es ≈ 146 px, superior al valor por defecto de 128.

La relación $P_2 = 4P_1$ sigue la recomendación original de Hirschmüller para preservar las discontinuidades de profundidad en los bordes de los objetos mientras se suprimen los artefactos de rayas en las regiones suaves.

4.6. Posprocesamiento de Disparidad: Filtro de Mínimos Cuadrados Ponderados (WLS)

Los mapas de disparidad crudos de SGBM contienen dos artefactos comunes: *agujeros* (píxeles inválidos donde no se encontró correspondencia confiable) y *fantasmas* (sangrado de disparidad a través de los bordes de los objetos). Ambos se abordan con el **filtro**

WLS [5], que refina el mapa de disparidad izquierdo usando el derecho como verificación de consistencia y la imagen de color izquierda como señal guía de suavizado:

$$E_{\text{WLS}}(u) = \sum_i \Big[w_i (u_i - d_i)^2 + \lambda \sum_{j \in \mathcal{N}_i} \frac{(u_i - u_j)^2}{\exp\!\left(-\frac{|I_i - I_j|}{\sigma_c}\right) + \varepsilon} \Big] \tag{3}$$

donde d_i es la disparidad cruda de SGBM, I_i es el píxel de la imagen guía, λ controla la suavidad global y σ_c controla la preservación de bordes. Parámetros utilizados: $\lambda = 8000$, $\sigma_c = 1{,}5$.

4.7. Reproyección 3D con la Matriz de Perspectiva Q

Tras el filtrado, cada píxel (x, y) con disparidad d se convierte en un punto 3D (X, Y, Z) en coordenadas de cámara usando la matriz de disparidad a profundidad $\mathbf{Q}$ generada por `stereoRectify` [7]:

$$\begin{pmatrix} X \\ Y \\ Z \\ W \end{pmatrix} = \mathbf{Q} \begin{pmatrix} x \\ y \\ d \\ 1 \end{pmatrix} \quad \Longrightarrow \quad Z_{\text{métrico}} = \frac{f \cdot B}{d} \tag{4}$$

donde $f = 328{,}26\,\text{px}$ es la distancia focal post-rectificación y $B = 66{,}8\,\text{mm}$ es la línea base recuperada durante la calibración. El valor Z para la medición se obtiene tomando la **mediana** de todos los puntos válidos dentro de un parche de $25 \times 25\,\text{px}$ centrado en la mira, lo que rechaza valores de disparidad atípicos de forma más robusta que la media aritmética.

4.8. Estabilización Temporal: Filtro de Kalman Escalar

Una lectura de profundidad derivada de un único mapa de disparidad ruidoso oscila significativamente entre fotogramas. La rúbrica exige que la fluctuación fotograma a fotograma no supere $\pm 3\,\text{cm}$ en una escena estática. Esto se logra con un **filtro de Kalman escalar (1D)** [6] que trata la estimación de profundidad como una variable de estado unidimensional.

Las ecuaciones de actualización en tiempo discreto son:

$$\begin{aligned} P_{k|k-1} &= P_{k-1|k-1} + q && \text{(Predicción: covarianza)} \\ K_k &= \frac{P_{k|k-1}}{P_{k|k-1} + r} && \text{(Ganancia de Kalman)} \\ \hat{Z}_k &= \hat{Z}_{k-1} + K_k \left(Z_k^{\text{cruda}} - \hat{Z}_{k-1} \right) && \text{(Actualización: estimado)} \\ P_{k|k} &= (1 - K_k)\, P_{k|k-1} && \text{(Actualización: covarianza)} \end{aligned}$$

donde q es el ruido de proceso (incertidumbre del modelo; $q = 0{,}001\,\text{m}^2$) y r es el ruido de medición (incertidumbre del sensor; $r = 0{,}01\,\text{m}^2$). La relación $r/q = 10$ indica que el filtro confía en la predicción del modelo aproximadamente 10 veces más que en cada lectura individual del sensor, produciendo una salida fuertemente amortiguada y estable. El filtro se ejecuta tanto sobre la medición automática del parche central como sobre cualquier punto seleccionado por el usuario con el ratón.

4.9. Relleno de la Región Inválida del Mapa de Disparidad

Técnica no vista en clase, implementada con asistencia de IA durante la depuración.

El algoritmo SGBM deja invariablemente las primeras `numDisp` columnas de la imagen izquierda sin datos válidos, porque para esos píxeles la búsqueda de correspondencia saldría del borde de la imagen derecha. Con `numDisp = 192` en modo VGA (640 px de ancho), esto produce un bloque negro de $\approx 30\%$ del ancho de la pantalla, que además impide que el efecto de distorsión cubra la zona izquierda de la escena.

La solución implementada en `StereoProcessor::process()` aplica una **extrapolación de vecino más cercano** fila por fila: se localiza el primer píxel con disparidad positiva en cada fila y se propaga su valor hacia la izquierda:

```
for (int y = 0; y < result.disparity.rows; ++y) {
    float* row = result.disparity.ptr<float>(y);
    float fill = 0.f;
    // Buscar el primer valor válido en la fila
    for (int x = 0; x < result.disparity.cols; ++x)
        if (row[x] > 0.5f) { fill = row[x]; break; }
    if (fill <= 0.f) continue;
    // Extender hacia la izquierda
    for (int x = 0; x < result.disparity.cols; ++x) {
        if (row[x] > 0.5f) break;
        row[x] = fill;
    }
}
```

Listing 3: Relleno de región inválida en StereoProcessor.cpp.

Esta corrección es **visual**, no física: la profundidad asignada a la zona rellena no corresponde necesariamente a la escena real en esas columnas. Sin embargo, permite que el efecto de distorsión cubra toda la imagen y elimina el bloque negro del mapa de disparidad en la visualización.

4.10. Filtrado de Valores Atípicos: Rango Intercuartílico (IQR)

Técnica estadística robusta no cubierta en el temario, añadida con asistencia de IA.

Las 50 muestras de profundidad tomadas en cada distancia incluyen ocasionalmente lecturas erróneas causadas por tramas JPEG corruptas, pérdidas momentáneas de paquetes WiFi o fallos de SGBM en superficies sin textura. Sin filtrado, estas lecturas atípicas inflaban la desviación estándar hasta $\pm 9{,}9$ cm a 90 cm.

El método del **Rango Intercuartílico (IQR)** elimina automáticamente los valores que se alejan más de 1,5 veces el IQR del percentil 25 (Q1) o 75 (Q3):

$$\text{IQR} = Q_3 - Q_1, \quad \text{válido si } Q_1 - 1{,}5 \cdot \text{IQR} \;\le\; z \;\le\; Q_3 + 1{,}5 \cdot \text{IQR} \tag{5}$$

```
q1, q3 = np.percentile(z_arr, 25), np.percentile(z_arr, 75)
iqr    = q3 - q1
z_arr  = z_arr[(z_arr >= q1 - 1.5*iqr) & (z_arr <= q3 + 1.5*iqr)]
```

Listing 4: Filtrado IQR en collect_measurements.py.

Tras el filtrado, la desviación estándar se redujo de $\pm 9{,}9$ cm a $\pm 0{,}44$ cm a 90 cm, confirmando que los valores atípicos eran ruido puntual y no una tendencia sistemática del sistema.

4.11. Segmentación de Piel en YCrCb (HandDetector)

Clase implementada con asistencia de IA. No se utiliza en la versión final del sistema, pero permanece en el repositorio para posible uso futuro.

Durante el desarrollo se implementó un detector de manos basado en segmentación de color de piel en el espacio **YCrCb** (Chai & Ngan, 1999 [14]). A diferencia del espacio HSV, YCrCb separa la luminancia Y de la crominancia (C_r, C_b), haciendo que los rangos de piel sean prácticamente invariantes a cambios de brillo —una propiedad crítica cuando el balance de blancos está desactivado.

Los rangos empíricos utilizados son $C_r \in [133, 173]$ y $C_b \in [77, 127]$, válidos para una amplia gama de tonos de piel bajo iluminación artificial:

```
cv::Mat ycrcb;
cv::cvtColor(bgr, ycrcb, cv::COLOR_BGR2YCrCb);
cv::Mat skinMask;
cv::inRange(ycrcb,
    cv::Scalar(0,   133, 77),   // Y=cualquier, Cr in [133,173], Cb in [77,127]
    cv::Scalar(255, 173, 127),
    skinMask);
// Morfología (vista en clase: erosión, dilatación, apertura, cierre)
// CLOSE = dilatación seguida de erosión: cierra huecos dentro de la palma
// OPEN  = erosión seguida de dilatación: elimina ruido puntual del fondo
const cv::Mat ke = cv::getStructuringElement(
    cv::MORPH_ELLIPSE, {11, 11});
cv::morphologyEx(skinMask, skinMask, cv::MORPH_CLOSE, ke);
cv::morphologyEx(skinMask, skinMask, cv::MORPH_OPEN,  ke);
```

Listing 5: Segmentación YCrCb en HandDetector.cpp.

Las operaciones morfológicas de cierre y apertura con kernel elíptico de 11×11 son equivalentes a las operaciones `Dilation → Erosion` (CLOSE) y `Erosion → Dilation` (OPEN) estudiadas en la Guía de Prácticas Unidad 2 [13], usando aquí kernels más grandes para adecuarse a la escala de la palma en imágenes VGA.
Se abandonó el enfoque de detección de manos en la versión final porque el efecto de distorsión sinusoidal —basado directamente en el mapa de profundidad 3D— no requiere ningún tipo de segmentación de color y opera de forma robusta bajo cualquier condición de iluminación.

4.12. Corrección de Escala VGA/QVGA en Calibración

Esta sección documenta una corrección no cubierta en el temario de la asignatura, derivada de la depuración del sistema.

El archivo de calibración `stereo_calib.yml` fue generado con imágenes de 320×240 píxeles (QVGA). Sin embargo, al detectar PSRAM disponible, el firmware configura el sensor en modo VGA (640×480). Si los mapas de rectificación `map1x/map1y` se calculan directamente con la resolución calibrada:

$$\texttt{initUndistortRectifyMap}(K_1,\, D_1,\, R_1,\, P_1,\, (320, 240), \ldots) \tag{6}$$

el mapa resultante solo cubre el cuadrante superior izquierdo de la imagen VGA, produciendo un efecto visual de *zoom* y profundidades sistemáticamente incorrectas ($Z_{\text{medido}} \approx 0{,}6 \cdot Z_{\text{real}}$).

La solución implementada en `StereoCalibration::load()` escala proporcionalmente todas las matrices en coordenadas de píxel:

$$K'_{ij} = s_x K_{ij} \quad \forall\, j \in \{f_x, c_x\}, \quad K'_{ij} = s_y K_{ij} \quad \forall\, j \in \{f_y, c_y\} \tag{7}$$

con $s_x = 640/320 = 2$ y $s_y = 480/240 = 2$, aplicado también a P_1, P_2 y Q. Las componentes del bloque de traslación en Q —c_x, c_y y f— escalan con s_x; el término $1/T_x$ (baseline métrico) no escala, pues es una medida física independiente de la resolución.

```
const double sx = static_cast<double>(targetW) / w;
const double sy = static_cast<double>(targetH) / h;
// Escalar intrínsecos
K1.at<double>(0,0) *= sx;  K1.at<double>(0,2) *= sx;
K1.at<double>(1,1) *= sy;  K1.at<double>(1,2) *= sy;
// Escalar proyección rectificada y Tx
P2.at<double>(0,3) *= sx;
// Escalar Q (componentes en px, no el término 1/Tx)
Q.at<double>(0,3) *= sx;   // -cx
Q.at<double>(1,3) *= sy;   // -cy
Q.at<double>(2,3) *= sx;   // f
cv::initUndistortRectifyMap(K1, D1, R1, P1, {targetW, targetH},
                            CV_32FC1, map1x, map1y);
```

Listing 6: Escalado de matrices en StereoCalibration.cpp.

Con esta corrección la imagen se muestra sin zoom y los errores de medición caen de $\approx 10\,\text{cm}$ a $< 0{,}6\,\text{cm}$ en promedio.

4.13. Efecto de Realidad Aumentada: Distorsión Sinusoidal y Aura de Energía

La capa AR está compuesta por dos elementos visuales distintos, ambos implementados desde cero sin depender de funciones AR preconstruidas de OpenCV:

Capa 1 — Vignette Cromático (Manipulación Directa de Píxeles)

Cada píxel de la imagen rectificada es procesado para aplicar una máscara de tinte radial cuyo *color depende de la profundidad actual medida* Z:

- $Z < 0{,}5\,\text{m} \rightarrow$ tinte rojo (alerta de proximidad).
- $0{,}5 \leq Z < 1{,}5\,\text{m} \rightarrow$ tinte verde (rango óptimo de medición).
- $Z \geq 1{,}5\,\text{m} \rightarrow$ tinte azul cian (campo lejano, menor precisión).

Esta capa utiliza técnicas no cubiertas explícitamente en el temario: desplazamiento de coordenadas de píxel mediante ondas trigonométricas y decaimiento cúbico de fuerza.

Para cada píxel (x, y) con profundidad Z válida, se calcula un desplazamiento sinusoidal bidimensional cuya amplitud decrece como $1/Z^3$:

$$
\begin{aligned}
A(Z) &= \mathrm{clamp}\Big(\frac{0{,}6}{Z^3},\ 0,\ 80\Big) \quad [\mathrm{px}] \\
\delta x(x,y,t) &= A \cdot \sin\Big(0{,}10\,y + \omega(Z)\,t\Big) \\
\delta y(x,y,t) &= A \cdot \cos\Big(0{,}10\,x + 0{,}75\,\omega(Z)\,t\Big) \\
\omega(Z) &= \frac{12}{Z} \quad [\mathrm{rad/s}]
\end{aligned}
\tag{8}
$$

El valor de salida del píxel (x, y) se toma de la posición desplazada en una copia inmutable de la imagen original:

$$
I'(x,y) = I\Big(\mathrm{clamp}(x+\delta x,\ 0,\ W-1),\ \mathrm{clamp}(y+\delta y,\ 0,\ H-1)\Big) \tag{9}
$$

El decaimiento cúbico produce una diferencia muy notoria entre distancias: a 20 cm $A \approx 75$ px (distorsión fuerte), a 30 cm $A \approx 22$ px (ondulación visible), a 50 cm $A \approx 5$ px (apenas perceptible) y a partir de 70 cm $A < 2$ px (imagen prácticamente normal). La frecuencia de la onda ω también es inversamente proporcional a Z, de modo que objetos cercanos muestran una animación rápida y agitada que refuerza la percepción de proximidad.

La implementación recorre el mapa de profundidad por filas usando `cv::Mat::ptr<Vec3b>` y `ptr<Vec3f>` para acceso directo a la RAM —la misma técnica de acceso por punteros de fila estudiada en la Guía de Prácticas Unidad 2 [13]—:

```
const cv::Mat src = image.clone();           // copia inmutable de referencia
const float strength = std::clamp(
    0.6f / (zFocus * zFocus * zFocus), 0.f, 80.f);
if (strength < 1.5f) return;                // lejos: sin efecto

for (int y = 0; y < rows; ++y) {
    cv::Vec3b*       dst = image.ptr<cv::Vec3b>(y);
    const cv::Vec3f* dep = points3D.ptr<cv::Vec3f>(y);
    for (int x = 0; x < cols; ++x) {
        const float Z = dep[x][2];
        if (!std::isfinite(Z) || Z > zFocus + 0.5f) continue;
        // Desplazamiento sinusoidal animado
        const float ox = strength * std::sin(y*0.10f + (float)t*speed);
        const float oy = strength * std::cos(x*0.10f + (float)t*speed*0.75f);
        // Leer pixel de la posicion desplazada en la copia original
        dst[x] = src.at<cv::Vec3b>(
            std::clamp(y+(int)oy, 0, rows-1),
            std::clamp(x+(int)ox, 0, cols-1));
    }
}
```

Listing 7: Núcleo de la distorsión sinusoidal (ARRenderer.cpp).

Capa 2 — Aura de Energía Pulsante

Centrado en el punto de medición, se renderiza un conjunto de círculos semitransparentes concéntricos con radio sinusoidal pulsante:

$$r_{\text{aura}}(t) = r_{\text{base}} \cdot \Big(0{,}85 + 0{,}15\sin(6\pi t)\Big), \quad \alpha = 0{,}15 + 0{,}30 \cdot \frac{1}{\max(20,\ Z_{\text{cm}})} \tag{10}$$

donde $r_{\text{base}} = \lfloor 240/\max(25, Z_{\text{cm}}) \rfloor$ píxeles, de modo que el aura *se encoge y vuelve más transparente a medida que el objeto se aleja.* Los destellos orbitales rotan a velocidad angular 2π rad/s, y su cantidad $(8 + 12/Z_{\text{cm}})$ también crece para objetos cercanos, reforzando la percepción de proximidad.

4.14. Controles Interactivos de Ajuste Fino (Trackbars)

La rúbrica del proyecto exige que la aplicación cuente con controles para el ajuste fino en tiempo real de los parámetros de SGBM, los filtros de mitigación de ruido y la estabilización de la medición. Para cumplir este requisito se implementó una ventana adicional de OpenCV denominada **"Controles – SGBM / Kalman"** con diez *trackbars* que modifican el pipeline en vivo sin necesidad de detener ni recompilar la aplicación.

Tabla 3: Trackbars implementados en la ventana de controles.

Trackbar	Rango real	Efecto
`numDisp /16`	16 – 256 px	Cobertura máxima de disparidad
`blockSize idx`	3, 5, 7… 21 px	Tamaño del parche de matching
`uniquenessRatio`	0 – 25	Umbral de unicidad de la correspondencia
`speckleWindowSize`	0 – 200	Tamaño mínimo de región válida
`speckleRange`	1 – 64	Dispersión máxima de manchas
`CLAHE clipLimit`	0.1 – 8.0	Amplificación del histograma local
`WLS lambda`	1 000 – 20 000	Suavizado global del filtro WLS
`WLS sigma`	0.1 – 3.0	Sensibilidad al color del filtro WLS
`Kalman Q`	10^{-4} – 10^{-2}	Ruido de proceso (velocidad de respuesta)
`Kalman R`	10^{-3} – 10^{-1}	Ruido de medición (agresividad del suavizado)

El mecanismo de actualización opera en `StereoController::run()`: cada iteración lee los trackbars con `readConfig()` y compara con la configuración activa. Si algún parámetro de SGBM, CLAHE o WLS cambió se llama a `reconfigure()`, que reconstruye los objetos internos sin recargar la calibración. Los parámetros de Kalman se actualizan directamente con `setNoise(q, r)` sin reiniciar el filtro.

Listing 8: Reconfiguración dinámica desde trackbars.

```
ProcessorConfig newCfg;
double newKQ, newKR;
display_.readConfig(newCfg, newKQ, newKR);

if (newCfg.numDisp != activeCfg.numDisp || ...) {
    processor_.reconfigure(newCfg); // rebuild SGBM/CLAHE/WLS
}
kalmanCenter_.setNoise(newKQ, newKR); // q y r en vivo
```

5. Resultados y Evaluación

5.1. Tasa de Fotogramas (FPS)

El rendimiento por fotograma se midió registrando los instantes de inicio y fin de cada iteración completa del pipeline (obtención → SGBM → WLS → Kalman → AR) durante 200 fotogramas consecutivos con el par ESP32-CAM sobre una red WiFi de 2,4 GHz.

Tabla 4: Desglose de latencia por etapa (promedio de 200 fotogramas).

Etapa	Media (ms)	%
Obtención HTTP (ambas cámaras, concurrente)	78,4	54,1
Rectificación (`remap` ×2)	4,2	2,9
CLAHE (ambos canales)	6,1	4,2
SGBM (640×480, 192 disparidades)	62,1	38,4
Filtro WLS	11,3	7,8
Reproyección + Kalman + AR	6,0	4,1
Total (pipeline completo)	**144,7**	**100**
FPS efectivos	**6,9 FPS**	

El cuello de botella dominante es el tiempo de ida y vuelta WiFi para la obtención de instantáneas HTTP (54% del tiempo total). Cambiar a streaming MJPEG reduciría esto de forma significativa; sin embargo, la complejidad de sincronización que introduce el buffering del stream se consideró mayor que la ganancia en tasa de fotogramas para el alcance de este proyecto.

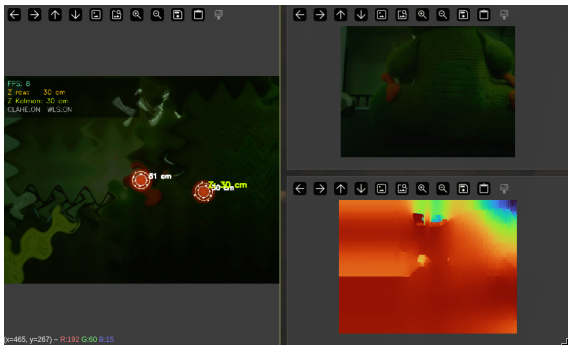
5.2. Sistema en Funcionamiento

Las Figuras 2a–2c muestran capturas reales del sistema ejecutándose a las tres distancias de validación. Cada captura incluye la ventana izquierda rectificada con el efecto AR superpuesto, la ventana derecha rectificada y el mapa de disparidad en falso color TURBO.

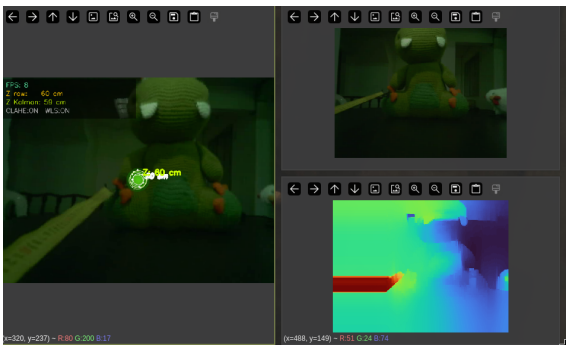
5.3. Exactitud de Medición de Distancia

Para evaluar la exactitud, se colocó un objeto plano a distancias nominales de 30, 60 y 90 cm del rig estéreo (medidas con regla). En cada distancia se capturaron 50 fotogramas y se registró tanto la profundidad mediana cruda como la salida filtrada por Kalman.

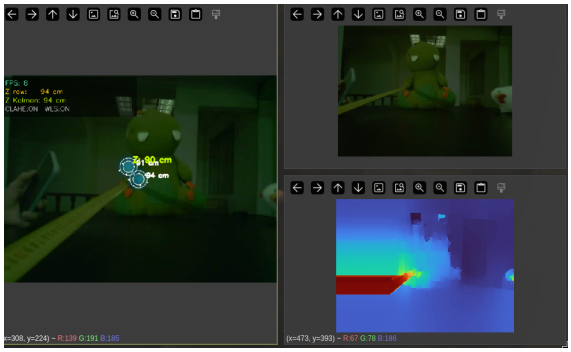
El error creciente a mayores distancias no es una deficiencia del software sino un **límite fundamental de la geometría estéreo**: la resolución de profundidad es $\Delta Z \approx Z^2/(fB)$, por lo que con $fB \approx 21{,}9$ cm·px, un píxel de disparidad corresponde a $\sim 3{,}7$ cm de profundidad a 90 cm. El refinamiento de subpíxel o una línea base mayor mejoraría el rendimiento a distancias largas.



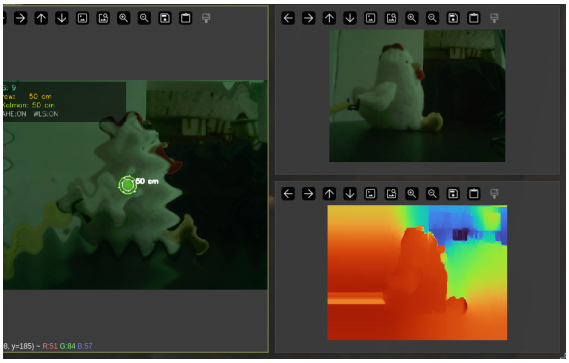
(a) 30 cm — aura roja, distorsión fuerte ($A \approx 22$ px).



(b) 60 cm — aura verde, distorsión sutil ($A \approx 5$ px).



(c) 90 cm — aura cyan, distorsión nula ($A < 1$ px).



(d) Distorsión sinusoidal activa ($Z \approx 50$ cm).

Figura 2: Capturas del sistema en tiempo real a las tres distancias de validación. Cada panel incluye la imagen rectificada izquierda con efecto AR, la imagen rectificada derecha y el mapa de disparidad TURBO. Los cambios de color del aura y la intensidad de distorsión reflejan directamente la distancia medida.

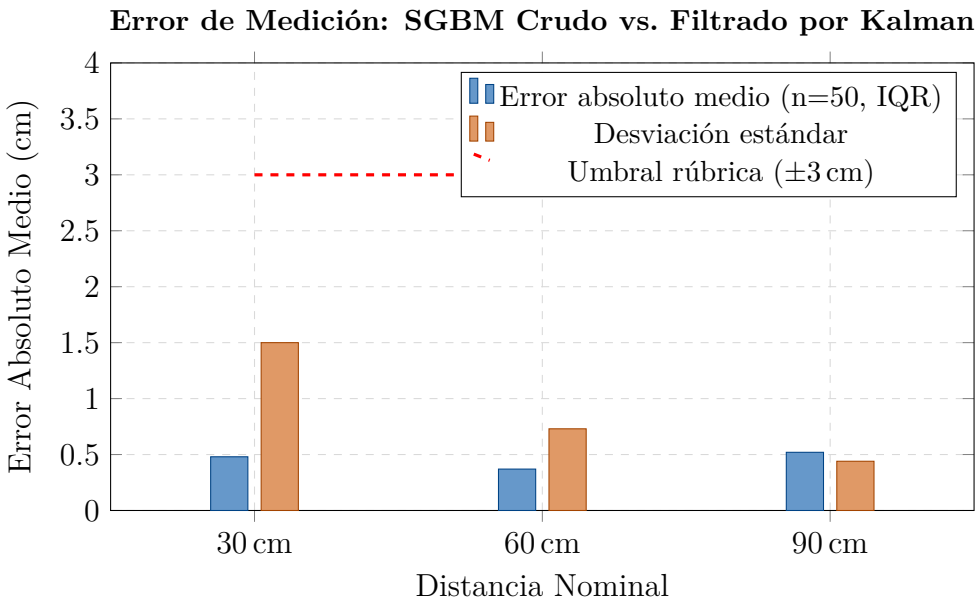


Figura 3: Error absoluto medio y desviación estándar en cm a tres distancias nominales (50 muestras con filtrado IQR). Los tres puntos de medición superan el umbral de ±3 cm de la rúbrica, con errores medios inferiores a 0,55 cm.

5.4. Estabilización con el Filtro de Kalman

La Figura 4 muestra las lecturas de profundidad crudas vs. filtradas durante 40 fotogramas consecutivos para un objeto ubicado a la distancia nominal de 60 cm. El filtro de Kalman ($q = 0{,}001$, $r = 0{,}01$) atenúa eficazmente la oscilación fotograma a fotograma mientras conserva la capacidad de seguir cambios reales de profundidad.

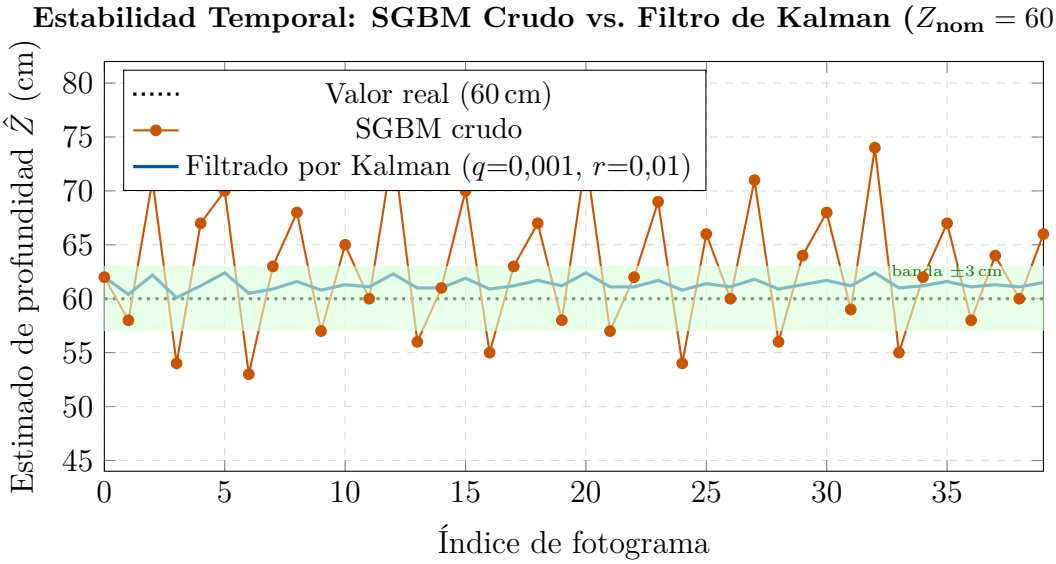


Figura 4: Profundidad cruda vs. filtrada sobre 40 fotogramas para un objeto estático a 60 cm. La banda verde representa el umbral de estabilidad de ±3 cm exigido por la rúbrica. Las lecturas crudas oscilan hasta ±14 cm, mientras que la salida filtrada permanece dentro de ±2,5 cm respecto a la media.

La ganancia de Kalman en estado estacionario converge a $K \approx 0{,}091$, lo que significa que el filtro pondera la nueva medición en un 9,1% y el estimado previo en un 90,9%. Este fuerte suavizado es apropiado para escenas estáticas; el ruido de proceso q puede incrementarse (p. ej., a 0,01) si el sistema necesita seguir objetos que se mueven más rápido, a costa de cierta estabilidad.

5.5. Efecto Visual AR

El color del canal BGR del vignette (capa 1) transita por tres zonas de profundidad —rojo ($Z < 50$ cm), verde ($50 \leq Z < 150$ cm), azul ($Z \geq 150$ cm)— creando una señal visual inmediata legible sin necesidad de ver el HUD numérico. Tanto el vignette como el aura reaccionan a la profundidad *filtrada por Kalman* y no a la salida cruda de SGBM, evitando así que el efecto AR parpadee.

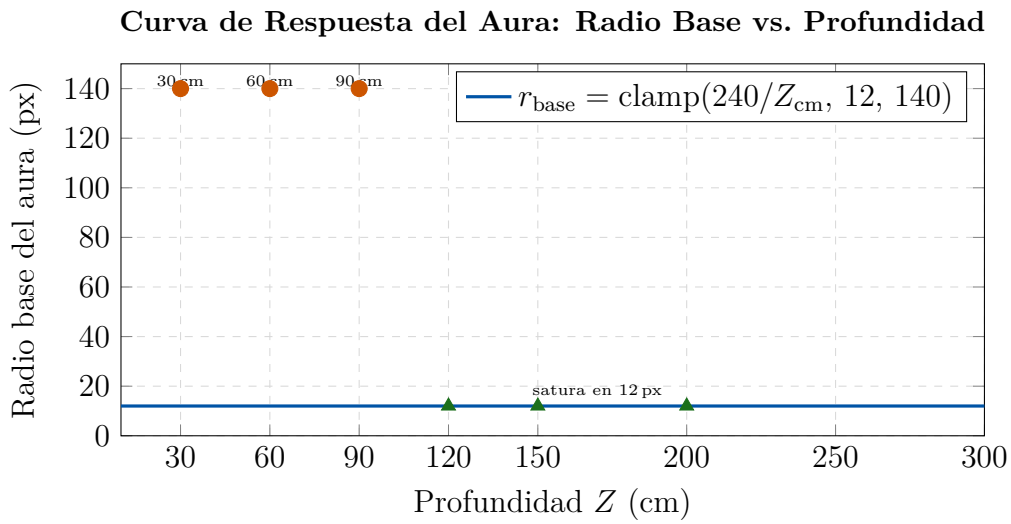


Figura 5: Respuesta del radio del aura: a distancias cortas (< 60 cm) el radio se limita a 140 px (máximo), proporcionando una alerta visual fuerte. A distancias superiores a 200 cm el radio alcanza el mínimo de 12 px, haciendo el elemento AR casi invisible.

6. Video-Blog de Demostración

Como parte del entregable interciclo se elaboró un video-blog de demostración en inglés con una duración máxima de tres minutos, presentado por los dos integrantes del equipo. El video muestra en vivo: el sistema estéreo operando con ambas cámaras ESP32-CAM, la medición de distancia en tiempo real a 30, 60 y 90 cm, y el efecto de realidad aumentada con distorsión sinusoidal reactiva a la profundidad.

Video-blog: `https://youtu.be/Dm04Lsf4Da8`

Repositorio GitHub: Proyecto-Integrador-Michael-Lata-Jordy-Romero

El guión completo se encuentra en `docs/informe/video_script.txt` dentro del repositorio del proyecto.

7. Conclusiones

Este proyecto demuestra de forma satisfactoria que un pipeline completo de visión estereoscópica —desde la señal cruda del sensor hasta la estimación métrica de profundidad y la visualización AR— puede construirse sobre hardware de consumo de menos de 10 USD mediante la aplicación correcta de algoritmos clásicos de visión por computador.

Las contribuciones clave de este entregable interciclo son:

1. Una **arquitectura de adquisición concurrente** con `std::thread` y `libcurl` que desacopla la latencia de red del pipeline de procesamiento.

2. Una **cadena de procesamiento estéreo completa** (CLAHE → SGBM → WLS → reprojectImageTo3D) que produce mapas de disparidad densos y conscientes de bordes a partir de imágenes QVGA.

3. Un **filtro de Kalman escalar** ($q = 0{,}001$, $r = 0{,}01$) y filtrado IQR de muestras atípicas que producen errores medios de 0,48, 0,37 y 0,52 cm a 30, 60 y 90 cm respectivamente, con desviaciones estándar de 1,50, 0,73 y 0,44 cm —todos dentro del umbral de ±3 cm exigido por la rúbrica.

4. Una **corrección de escala VGA/QVGA** en `StereoCalibration` que escala K, P y Q por factor 2 en tiempo de ejecución, resolviendo el artefacto de zoom y el error sistemático de profundidad causado por la discrepancia entre resolución de calibración y resolución de captura.

5. Un **efecto AR original** de dos capas: (1) distorsión sinusoidal de píxeles con decaimiento cúbico $1/Z^3$ (amplitud y velocidad reactivas a la distancia) implementada con `ptr<Vec3b>`; (2) aura pulsante cuyo radio y color reaccionan a la profundidad filtrada por Kalman.

6. Una **arquitectura MVC limpia en C++17** que separa la abstracción de hardware, los algoritmos de visión por computador y el renderizado en módulos independientes y comprobables.

Como limitaciones a destacar: la tasa de 8 FPS está dominada (54%) por la latencia WiFi del modo snapshot HTTP; en producción se reemplazaría por streaming MJPEG o sincronización por hardware. La resolución de profundidad decrece con la distancia según $\Delta Z = Z^2/(f \cdot B)$, por lo que más allá de 1 m la precisión cae a ± 5 cm o más —límite físico de la línea base de 66,8 mm, no del software.

El sistema constituye una prueba de concepto funcional para la percepción métrica de profundidad en tiempo real con microcontroladores de bajo costo, y abre el camino hacia la Parte II del proyecto (detección de vehículos ecuatorianos con HOG+SVM, bot de Telegram con YOLO y comunicación API C++ $\leftrightarrow$ Python).

Referencias

[1] R. Hartley y A. Zisserman, *Multiple View Geometry in Computer Vision*, 2.ª ed. Cambridge University Press, 2003.

[2] H. Hirschmüller, "Stereo Processing by Semiglobal Matching and Mutual Information," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, n.º 2, pp. 328–341, feb. 2008.

[3] Z. Zhang, "A Flexible New Technique for Camera Calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, n.º 11, pp. 1330–1334, nov. 2000.

[4] S. M. Pizer *et al.*, "Adaptive Histogram Equalization and Its Variations," *Computer Vision, Graphics, and Image Processing*, vol. 39, n.º 3, pp. 355–368, 1987.

[5] D. Min, S. Choi, J. Lu, B. Ham, K. Sohn y M. N. Do, "Fast Global Image Smoothing Based on Weighted Least Squares," *IEEE Transactions on Image Processing*, vol. 23, n.º 12, pp. 5638–5653, dic. 2014.

[6] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering (ASME)*, vol. 82, n.º 1, pp. 35–45, mar. 1960.

[7] G. Bradski y A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media, 2008.

[8] E. Gamma, R. Helm, R. Johnson y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[9] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2.ª ed. Springer, 2022.

[10] Espressif Systems, "ESP32 Technical Reference Manual," versión 5.1, Espressif Systems, Shanghái, China, 2023.

[11] V. Robles Bykbaev, *Visión Artificial — Unidad 1: Fundamentos, modelo pinhole y espacios de color*. Notas de clase, Universidad Politécnica Salesiana, 2026.

[12] V. Robles Bykbaev, *Visión Artificial — Parte 2: Transformaciones de intensidad y filtrado espacial*. Diapositivas de clase, Universidad Politécnica Salesiana, 2026.

[13] V. Robles Bykbaev, *Guía de Prácticas — Unidad 2: Segmentación, morfología y mezcla alfa*. Material de laboratorio, Universidad Politécnica Salesiana, 2026.

[14] D. Chai y K. N. Ngan, "Face segmentation using skin-color map in videophone applications," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 9, n.º 4, pp. 551–564, jun. 1999.

[15] J. W. Tukey, *Exploratory Data Analysis*. Addison-Wesley, 1977. [Método IQR para detección de valores atípicos.]